

Question_9:

Code:

```
1 // There are n rooms and m directed teleports. Each teleporter can be used at
2 // most once. Find the maximum number of days Teddy can travel from room 1 to n.
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6
7 #define MAXN 505
8
9 int capacity[MAXN][MAXN];
10 int adj[MAXN][MAXN];
11 int deg[MAXN];
12 int parent_node[MAXN];
13 int visited[MAXN];
14
15 int bfs(int n, int s, int t)
16 {
17     memset(visited, 0, sizeof(visited));
18     int queue[MAXN];
19     int head = 0, tail = 0;
20     queue[tail++] = s;
21     visited[s] = 1;
22     parent_node[s] = -1;
23     while (head < tail)
24     {
25         int u = queue[head++];
26         if (u == t)
27             return 1;
28         for (int i = 0; i < deg[u]; i++)
29         {
30             int v = adj[u][i];
31             if (!visited[v] && capacity[u][v] > 0)
32             {
33                 visited[v] = 1;
34                 parent_node[v] = u;
35                 queue[tail++] = v;
36             }
37         }
38     }
```

```
39     return 0;
40 }
41 int main()
42 {
43     int n, m;
44     if (cin >> n >> m)
45     {
46         for (int i = 0; i < m; i++)
47         {
48             int u, v;
49             cin >> u >> v;
50             capacity[u][v] = 1;
51             adj[u][deg[u]++] = v;
52             adj[v][deg[v]++] = u;
53         }
54         int max_days = 0;
55         while (bfs(n, 1, n))
56         {
57             max_days++;
58             int curr = n;
59             while (curr != 1)
60             {
61                 int prev = parent_node[curr];
62
63                 capacity[prev][curr] -= 1;
64                 capacity[curr][prev] += 1;
65                 curr = prev;
66             }
67         }
68         cout << max_days << "\n";
69         for (int d = 0; d < max_days; d++)
70         {
71             int path[MAXN];
72             int len = 0;
73             int curr = 1;
```

```
74     path[len++] = curr;
75     while (curr != n)
76     {
77         for (int i = 0; i < deg[curr]; i++)
78         {
79             int v = adj[curr][i];
80             if (capacity[v][curr] > 0)
81             {
82                 capacity[v][curr] = 0;
83                 curr = v;
84                 path[len++] = curr;
85                 break;
86             }
87         }
88     }
89     cout << len << "\n";
90     for (int i = 0; i < len; i++)
91     {
92         cout << path[i];
93         if (i != len - 1)
94             cout << " ";
95     }
96     cout << "\n";
97 }
98 }
99 return 0;
100 }
```

Output:

Test Case_1:

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
2
3
1 2 6
4
1 3 4 6
PS C:\Users\Prabin Yadav\
```

Test Case_2:

```
6 7
1 4
3 5
4 6
5 6
2 2
1 3
2 6
2
3
1 4 6
4
1 3 5 6
PS C:\Users\Prabin Yadav
```